

PROPERTIES

title Sign2Graph — Hackathon Brief

type project-documentation

status active

tags project sign2graph hackathon edtech computer-vision artificial-intelligence

created 2026-08-02

updated 2026-08-02

Sign2Graph Brief

Sign2Graph is an AI-powered spatial STEM learning platform that turns an expression written in the air into an interactive graph or a structured, narrated formula lesson.

Category: AI, computer vision, EdTech · **Platform:** Web · **Hardware:** Laptop and webcam · **Primary users:** Students, teachers, tutors, schools, and educational exhibits

1. The Problem

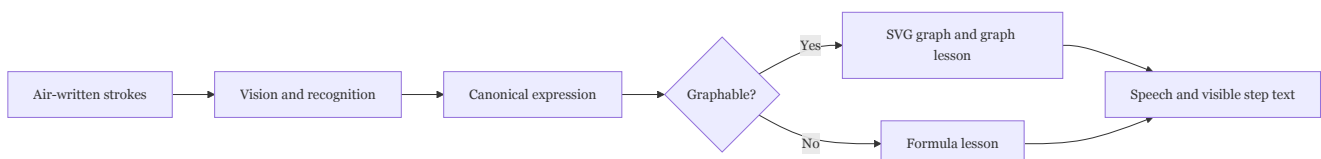
Most graphing tools show a final graph but provide limited explanation of how it was constructed. Static text can make abstract STEM relationships difficult to follow, while many spatial-learning systems require touchscreens, interactive boards, or AR hardware. Underfunded schools may not have that equipment, and teachers often switch between separate tools for graphing, explanation, narration, and presentation.

2. The Solution

Sign2Graph has two connected workflows:

- **Graph:** air-written expression → recognition → graphability check → graph → animated explanation → narration and visible step text
- **Other STEM:** scientific expression → recognition → subject context → structured formula lesson → narration and visible step text

For $y = (x - 2)^2 - 4$, the graph flow can begin with $y = x^2$, show the horizontal shift, then show the vertical shift and final curve. For $F = ma$, the expression is routed to the formula workspace instead of being forced onto a coordinate graph. Gemini proposes a structured scene plan; deterministic application code validates and renders the plan.



3. Live User Experience

The user enables the webcam, draws with a hand gesture, and releases the gesture. The app captures strokes, shows recognition feedback, and confirms the expression when the user selects **Add**. The expression is classified as graph or formula; a validated explanation plan is prepared; Graph or Others

mode opens; and the user can view, pause, replay, or advance the explanation without restarting the app. The webcam is optional after input: the rest of the interface has ordinary controls.

4. Why It Is Original

Handwriting recognition, graphing, gesture tracking, and AI tutoring are not individually new. The distinctive contribution is their integration into one spatial learning workflow: air-written STEM input, automatic routing, graph and non-graph explanation modes, AI-generated structured scenes, deterministic rendering, progressive graph construction, hand controls, browser narration, and synchronized on-screen step text.

During competitor and open-source research, we did not identify an available all-in-one platform combining this complete workflow. Sign2Graph is not positioned as a replacement for every specialist tool: Desmos and graphing calculators are stronger as mature graphing environments; handwriting tools focus on recognition; air canvases focus on spatial input; generic AI tutors focus on dialogue; presentation software focuses on authoring. Sign2Graph focuses on the missing transition from expression to visual construction.

5. Technical Effort and Architecture

User input → MediaPipe landmarks and stroke processing → Gemini or local recognition → canonical expression → validated scene schema → deterministic SVG/React renderer → animation, browser speech, and visible step text

The repository uses Next.js 15.3.2, React 19, TypeScript, `@mediapipe/tasks-vision` 1.0.1, ONNX Runtime Web with a bundled symbol model, `mathjs`, KaTeX, and browser `speechSynthesis`. Gemini 3.1 Flash-Lite is called server-side through `/api/recognize` and `/api/explore`; the key is never placed in client code. The app also stores sessions, gesture settings, narration preferences, and handwriting corrections locally.

AI output is not allowed to control the interface directly. Graph plans are limited to named actions such as `DRAW_BASE_GRAPH`, `TRANSFORM_GRAPH`, `DRAW_POINT`, and `HIGHLIGHT_GRAPH`; formula plans contain typed text, LaTeX, variables, assumptions, limitations, and steps. Length checks, allowed-value checks, expression checks, duplicate-ID checks, retry logic, and fallback plans contain malformed or unsafe output.

6. Accessibility and Educational Impact

The same concept can be represented visually (graph and transformations), textually (equations and explanations), spatially (air-writing and gestures), and auditorily (browser narration). The current interface displays step narration on screen while that step is spoken; this is caption-style visible text, not a separate subtitle file or export track.

Feedback from interviews with neurodivergent learners, including people with ADHD, dyslexia, and autism, and with learners who have attention difficulties, different processing speeds, or hearing impairments, influenced the emphasis on short explanation stages, replay and next-step controls, adjustable narration rate, reduced visual overload, and multiple representations. Relevant specialists were also consulted. These statements describe design input, not a medical claim or guaranteed outcome.

Because it needs no AR headset or interactive whiteboard, the prototype has a plausible path to classrooms, tutoring, self-study, museums, science demonstrations, and projector-based presentations. Future AR interfaces are an extension, not a current hardware requirement.

7. Project Condition

Capability	Status	Evidence or limitation
Webcam air-writing	Working within supported scope	MediaPipe hand landmarks, pinch/dwell stabilization, drawing, erasing, pointing, navigation, graph movement, and two-hand scaling are implemented. Requires browser camera permission and suitable lighting/tracking.
Recognition	Working within supported scope	Gemini image recognition is server-backed and returns LaTeX, canonical notation, confidence, and mode. Local geometric recognition and optional ONNX symbol candidates provide a fallback; recognition can be wrong and is shown for confirmation.
Graph rendering	Working	SVG rendering supports explicit $y = \dots$ sampling and implicit equalities through mathjs /contour sampling, with fit, pan, zoom, visibility, and multiple equations. Unsupported or malformed expressions produce no trace rather than an executable graph.
Graph explanation	Working within supported scope	Gemini plans are schema-validated; deterministic graph fallback covers the general parent-to-final flow and has an explicit translated-parabola path. AI plans can fail and fall back for graph mode.
Formula lessons	Working within supported scope	Others mode renders structured lessons with LaTeX, variables, assumptions, limitations, and steps. Verified local fallbacks cover selected physics formulas including $F = ma$, $V = IR$, and buoyant-force notation; other formulas depend on Gemini.
Narration and visible text	Working within supported scope	Browser speech synthesis is step-based with pause/replay/error reporting, and the active step is visible. Voice availability and autoplay/browser behavior vary.
Persistence and recovery	Working	Sessions and preferences are stored locally; loading, retry, missing-key, timeout, invalid-response, and unsupported-input states are handled. There is no account or cloud sync.
Evidence boundary	Partial	Automated tests cover core gesture, normalization, graph, and fallback logic. A production accuracy rate, user count, benchmark, deployment result, and formal accessibility study are not established in the repository.

The current prototype proves an end-to-end local interaction: webcam gesture → expression recognition/confirmation → graph or formula workspace → rendered explanation with optional narration and visible step text. It does not prove universal recognition accuracy, broad curriculum coverage, cloud scale, or formal learning outcomes.

8. Business and Investor Perspective

Students are the lesson users; teachers and tutors are presenters; schools, tutoring platforms, museums, and EdTech providers are possible paying customers. The value proposition is one interface that

combines input, graphing, explanation, narration, and presentation on common hardware while reducing the preparation needed for animated STEM demonstrations.

Possible models are a free student/demo tier, teacher subscription, school licensing, API or SDK licensing, paid curriculum/visualization packs, and museum or institutional deployments. The current webcam app validates the interaction model; scalability would come from more graph families and STEM subjects, multilingual narration/captions, saved lessons, teacher-authored libraries, LMS integration, and later AR or spatial-display clients. No revenue, market-size, partnership, or user-count claim is made here.

9. Why Sign2Graph Should Win

Originality: Existing tools can graph equations, recognize handwriting, track gestures, or generate explanations, but these features are usually separated across different products. Sign2Graph combines air-written STEM input, automatic classification, animated graph construction, multimodal formula explanations, gesture control, narration, and synchronized captions in one platform. The originality lies in turning a handwritten expression into a complete spatial learning experience that shows how a concept is built, rather than only presenting the final answer.

Effort: Sign2Graph required the integration of several technically demanding systems into one stable workflow. The project combines computer vision, hand tracking, gesture recognition, stroke processing, multimodal AI, mathematical parsing, graph rendering, animation, speech synthesis, captioning, and structured response validation. The main challenge was making these systems work together in real time while keeping the interface simple. AI generates a structured scene plan, while deterministic application logic renders the graph, explanation, narration, and captions to reduce failures and maintain consistency.

Impact: It can make construction and relationships more visible on hardware already found in many **classrooms**, tutoring settings, and exhibits, which makes it an important investment into accessible education future.

Project Condition: The prototype works end to end within a clearly defined scope, with local fallbacks and visible error states instead of pretending that recognition or AI is infallible.

Sign2Graph provides the missing step between seeing a formula and understanding how it works.

10. Judge FAQ

Why not use Desmos? Desmos is an excellent graphing tool; Sign2Graph addresses the preceding teaching step by turning an expression into a narrated, staged construction and also handling non-graph formulas.

Why gestures instead of a keyboard? Air-writing makes the expression itself the starting point and supports an interactive demonstration. Keyboard and pointer controls remain available for ordinary operation.

What happens when recognition is wrong? The interpreted expression is shown for confirmation. Gemini errors, timeouts, missing keys, malformed output, and unsupported expressions surface as recoverable UI states; local recognition and deterministic graph fallbacks cover some cases.

Why is AI necessary? AI helps interpret multimodal input and choose a structured explanation for formulas and graph transformations. Deterministic code still owns rendering, validation, and supported actions.

How is AI output kept reliable? Server routes constrain input, Gemini is asked for JSON, schemas and safe-expression checks validate the response, reusable plans must meet minimum step requirements, and

known local fallbacks exist.

Does it require AR glasses? No. The current prototype uses a standard laptop webcam; AR is a future client option.

Who would use or purchase it? Students use it; teachers, tutors, schools, tutoring platforms, museums, and EdTech providers could present, license, or integrate it.

What is fully working today? The web prototype covers webcam tracking, air-stroke capture, recognition paths, expression confirmation, graphing, selected formula lessons, gesture controls, local session storage, step navigation, browser narration, and visible step text.

What are the current limitations? Recognition is not guaranteed; camera and browser voice support vary; general formula lessons depend on Gemini; graph support is bounded by the parser/sampler; there is no account system, cloud sync, formal accuracy benchmark, or separate caption export.

What would funding be used for? Broader recognition and graph coverage, stronger testing and accessibility validation, multilingual and teacher-authored content, reliable deployment infrastructure, and integrations for schools and EdTech platforms.